

PROJECT REPORT

AI-Based Scene Understanding and Threat Detection Using Video Analytics

ThreatVision: a real-time system for person counting, gender estimation, activity recognition, interaction analysis, threat assessment and monitoring

Submitted by: _____

Guide: _____

Department / Institution: _____

September 2026

Abstract

Video surveillance is now everywhere, but human operators cannot watch dozens of feeds continuously, so incidents are often noticed only after they happen. This project presents **ThreatVision**, an end-to-end video analytics system that turns raw camera streams into a structured understanding of the scene and prioritised security alerts. Each frame is processed by a YOLOv8-pose network that detects people and their 17 body keypoints, and ByteTrack keeps a consistent identity for each person across frames. For every tracked person the system estimates gender (a face-based CNN and a full-body CNN with temporal voting) and recognises the activity (standing, walking, running, fighting, falling) from 30-frame skeleton sequences. Three activity recognisers are implemented and compared: an interpretable rule-based classifier, a bidirectional LSTM with attention, and a temporal convolutional network.

A scene-understanding module reasons about interactions and context. It detects physical altercations, chasing, loitering, restricted-zone intrusion, a person who stays down after a fall, group gathering, crowding, panic running, abandoned bags and presence during quiet hours. A threat assessment engine gives each situation an explainable 0–100 score and a severity level (LOW, MEDIUM, HIGH, CRITICAL), removes duplicate alerts and escalates situations that get worse. It then notifies security staff through a web dashboard (live video, statistics, alert feed, event history, analytics) and optionally by e-mail, webhook or Telegram. An evaluation framework measures detection accuracy, precision, recall, F1-score, AP@0.5, counting accuracy and processing speed for every algorithm, and the results are shown on the dashboard.

On the synthetic self-test scene the complete pipeline correctly raised all 18 scripted threat events, with no false alarms, at more than 300 analytics frames per second. On real COCO images the classical HOG+SVM baseline reached only 3.9 % recall, which shows why a deep detector is needed. The report describes the design, implementation, evaluation protocol and the steps needed to benchmark the system on real annotated footage.

Keywords: *video analytics, scene understanding, human activity recognition, pose estimation, YOLOv8, ByteTrack, LSTM, TCN, anomaly detection, threat assessment, intelligent surveillance.*

Table of Contents

(Right-click → Update Field in Microsoft Word to refresh page numbers.)

1. Introduction

1.1 Background and motivation

Closed-circuit television (CCTV) is used to protect campuses, transport hubs, shopping malls, hospitals and industrial sites. The number of cameras has grown much faster than the number of people available to watch them. Studies of operator attention show that vigilance drops sharply after about 20 minutes of monitoring, so most footage is only reviewed after an incident. Intelligent video analytics tries to reverse this by automatically understanding what is happening in the scene and bringing only the relevant situations to a human operator.

Detecting that a person is present is not enough. A person running for a bus and a person fleeing an assault look alike in a single frame. The difference lies in the **temporal context** (what happened before), the **interaction** with other people (is somebody chasing them?) and the **spatial context** (are they entering a restricted area?). This project therefore builds a layered system: perception (detection, tracking, pose), per-person attributes (gender, activity), scene-level reasoning (interactions and context) and decision support (threat scoring and alerting).

1.2 Objectives

1. Identify the number of persons in the scene and the (estimated) gender of each person.
2. Design a scene understanding module that recognises common human activities — walking, standing, running, fighting and falling — using deep learning techniques.
3. Analyse interactions and contextual information within a scene to identify suspicious or abnormal activities that may pose potential security threats.
4. Develop a real-time threat assessment and alert generation mechanism that classifies detected events by severity and notifies security personnel.
5. Design and implement a monitoring dashboard that shows live video feeds, detected activities, threat alerts and event history.
6. Evaluate the system with standard metrics — detection accuracy, precision, recall, F1-score and real-time processing speed — and display these metrics for every implemented algorithm on the dashboard.

1.3 Scope and contributions

- A modular, configurable pipeline in Python. Each stage (detector, gender model, activity model) can be swapped from the configuration file.
- Three activity recognisers built on one shared, scale-invariant 76-dimensional pose feature representation, so they can be compared fairly.
- A rule-based scene reasoning layer covering eleven kinds of suspicious situation, including pairwise (altercation, chase) and object-level (abandoned bag) reasoning.
- An explainable threat score: every alert lists the factors that produced its score.
- A self-contained web dashboard (no external CDN) with live streaming, push alerts, history, analytics and a metrics page.
- A reproducible evaluation toolkit, plus a scripted synthetic scene with exact ground truth for end-to-end self-testing.

2. Literature Review

2.1 Person detection and tracking

Early pedestrian detectors used hand-crafted features. The most influential is the Histogram of Oriented Gradients (HOG) with a linear SVM by Dalal and Triggs [1]. It works well for upright, unoccluded pedestrians but struggles with unusual poses, occlusion and small people. Convolutional one-stage detectors, starting with YOLO [2], now dominate real-time detection. The Ultralytics YOLOv8 family [3] adds anchor-free heads and a pose variant that predicts 17 COCO keypoints alongside each box, all in one forward pass. Multi-object tracking by detection links detections over time. SORT [4] uses a Kalman filter and Hungarian matching on IoU, and ByteTrack [5] improves robustness by also associating low-confidence detections, which matters under occlusion in crowds.

2.2 Gender estimation

Levi and Hassner [6] showed that a compact CNN trained on unconstrained face images (Adience) gives reliable age and gender estimates. Surveillance cameras, however, often capture faces that are too small or turned away. Pedestrian attribute recognition datasets such as PETA and PA-100K [7] instead label whole-body crops with attributes including gender, and lightweight backbones such as MobileNetV3 [8] make per-person classification feasible in real time. This project combines both: a face model when a face is visible and a body model otherwise.

2.3 Human activity recognition

Activity recognition can be based on appearance (RGB or optical flow, e.g. two-stream or 3D CNNs) or on skeletons. Skeleton-based methods are lighter, largely independent of clothing and background, and preserve privacy better. Sequences of joint coordinates can be modelled with recurrent networks such as the LSTM [9], with temporal convolutional networks (TCN), which Bai et al. [10] showed match or beat recurrent models on many sequence tasks, or with spatio-temporal graph networks (ST-GCN) [11]. Fall detection research uses datasets such as Le2i [12] and UR Fall [13], and violence detection uses RWF-2000 [14] and the Hockey Fight dataset.

2.4 Anomaly and threat detection

Sultani et al. [15] framed surveillance anomaly detection as weakly supervised multiple-instance learning on the UCF-Crime dataset. End-to-end anomaly scores are hard to explain to operators, so many practical systems combine learned perception with explicit, auditable rules: zones, dwell time, trajectories and object-owner relations such as left-luggage detection. ThreatVision follows this hybrid approach. Deep learning handles perception and activity, and transparent reasoning handles the scene context and severity.

3. System Architecture

Figure 1 shows the processing chain. A background thread reads frames from the configured source (webcam, video file, RTSP camera or the synthetic scene) and passes each frame through six stages. The results are sent to the alert manager and the dashboard. Stages communicate through small data classes (Detection, TrackState, SceneEvent, ThreatAlert), so any stage can be replaced without changing the others.

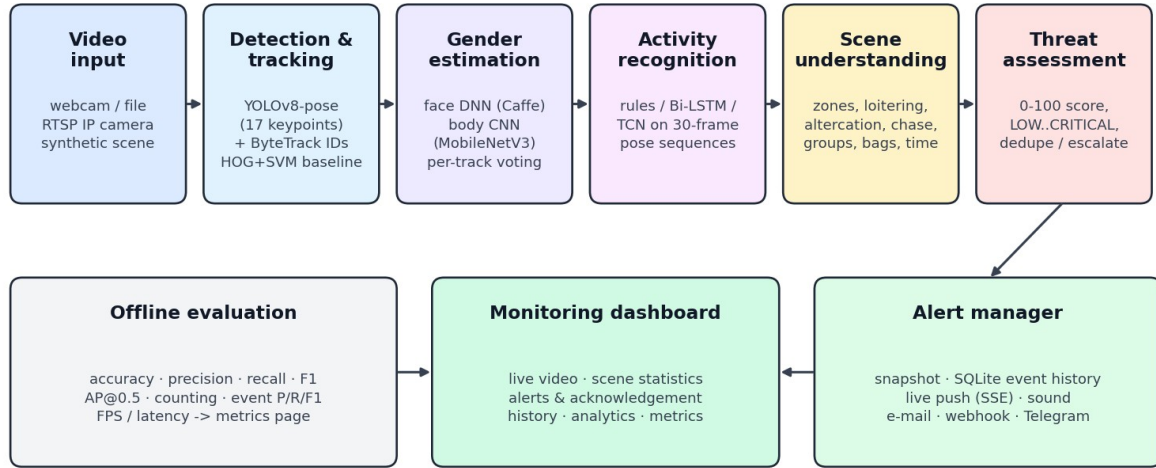


Figure 1 – ThreatVision architecture: the six-stage real-time pipeline, alerting, dashboard and offline evaluation.

Stage	Input	Output	Main technique
Detection & tracking	BGR frame	person boxes, 17 keypoints, track IDs; bags	YOLOv8-pose + ByteTrack (HOG+SVM + IoU tracker baseline)
Gender estimation	person crop	P(male), P(female) accumulated per track	Levi-Hassner face CNN / MobileNetV3 body CNN
Activity recognition	30-frame track history	activity label + confidence per person	rules / Bi-LSTM + attention / TCN
Scene understanding	all tracks, objects, zones, clock	SceneEvents (type, people, duration, zone)	geometric, temporal and relational rules
Threat assessment	SceneEvents	scored alerts, severity, reasons	additive scoring, dedupe, escalation
Alerting & dashboard	alerts, stats, frames	UI, notifications, SQLite history	Flask, MJPEG, Server-Sent Events

Table 1 – Pipeline stages.

4. Methodology

4.1 Person detection, counting and tracking

The primary detector is **YOLOv8-pose** (nano or small). It gives, for every person, a bounding box, a confidence score and 17 COCO keypoints (nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles), each with its own confidence. Detections above the operating threshold (0.40) are linked into tracks by **ByteTrack**. The number of persons in the scene is the number of confirmed tracks in the current frame, and the number of unique visitors is the number of distinct track IDs seen. A second YOLOv8 detector runs every fifth frame to find backpacks, handbags and suitcases for abandoned-object reasoning.

As a classical baseline, the system also has the **HOG + linear SVM** people detector from OpenCV, paired with a SORT-style IoU tracker using Hungarian assignment and constant-velocity prediction. Because the HOG detection window includes a margin around the body, its boxes are shrunk (15 % horizontally, 6 % vertically) to match tight ground-truth boxes.

4.2 Gender estimation

Two estimators are implemented and can be combined ('hybrid' mode):

- **Face DNN** – an OpenCV res10-SSD face detector searches the upper 45 % of the person box. If it finds a face larger than 16 px, the padded face is classified by the Levi-Hassner gender CaffeNet [6].
- **Body CNN** – a MobileNetV3-Small pretrained on ImageNet and fine-tuned on 256×128 full-body crops from PA-100K [7], with flipping, colour jitter, random crop and random erasing. It works even when the face is not visible.

Because a single frame can be misleading, the class probabilities are **summed over time for each track**. A label is shown only after at least 3 votes and an accumulated confidence of at least 0.60; otherwise the person is reported as 'undetermined'. Classification runs every third frame, staggered across tracks, and stops after 30 votes to save computation.

4.3 Activity recognition

Feature representation

For every tracked person the last $W = 30$ frames (about 1.2 s at 25 fps) are converted into a sequence of 76-dimensional feature vectors. All distances are divided by the person's reference standing height (75th percentile of recent box heights), so the features do not depend on camera distance or resolution:

Features	Dim.	Purpose
Keypoint (x, y) relative to box centre / body height	34	posture (arms raised, lying, crouching)
Keypoint velocities (body heights per second)	34	limb motion: strikes, kicks, leg swing
Box-centre velocity (vx, vy)	2	locomotion speed and direction; falling (vy)
Box aspect ratio w/h	1	upright vs. lying
Box height / reference height	1	sudden drop when falling
Torso angle (sin, cos) from vertical	2	orientation of the body
Nearest-person distance and its rate of change	2	interaction cue for fighting

Table 2 – Per-frame features used by all activity models.

Model A – Rule-based classifier (baseline)

A transparent decision procedure uses the same geometric quantities: (1) **falling** if the torso is more than 55° from vertical or the box is wider than tall (aspect > 1.15), or if the hips drop quickly while the box shrinks; (2) **fighting** if the fastest wrist moves faster than 2.6 body-heights/s relative to the hips, another person is within 1.1 body heights, the person is not travelling fast and the ankles are nearly still (which separates punching from arm swing while walking side by side); (3) **running** if the speed is at least 1.1 body-heights/s; (4) **walking** if at least 0.25; otherwise (5) **standing**. A majority vote over the last 7 predictions removes flicker.

Model B – Bidirectional LSTM with temporal attention

A LayerNorm + linear projection (76→128) is followed by a 2-layer bidirectional LSTM (128 units per direction, dropout 0.3). A learned attention layer weights the time steps, and the weighted sum passes

through a two-layer classification head. This lets the model focus on the decisive part of the window, such as the moment of impact in a fall.

Model C – Temporal Convolutional Network

Four residual blocks of dilated 1-D convolutions (kernel 3, dilations 1-2-4-8, 128 channels, batch norm, dropout 0.2) give a receptive field covering the whole 30-frame window. Global average and max pooling are concatenated before the classifier. TCNs are parallel over time, so they are usually faster than LSTMs at inference.

Training protocol

Pose sequences are extracted by running the same detector and tracker on labelled clips, with a sliding window of 30 frames and stride 10. Labels come from class folders or from frame-level annotation CSV files. The data is split **by source video** (70/15/15), so windows from one clip never appear in both training and test sets. Training uses AdamW (lr 1e-3, weight decay 1e-4), cosine annealing, class-balanced cross-entropy with label smoothing 0.05, gradient clipping, horizontal-flip and keypoint-jitter augmentation, and early stopping on validation macro-F1.

4.4 Scene understanding: interactions and context

The scene analyser takes all active tracks, detected objects, zone polygons and the wall clock, and produces **SceneEvents**. Each event has a type, the people involved, a duration, an optional zone and a confidence.

Event	Level of reasoning	Rule (defaults, configurable)
Restricted-zone intrusion	spatial context	foot point inside a restricted polygon; duration since entry
Loitering	temporal context	present ≥ 25 s while the position spread stays ≤ 3 body heights
Physical altercation	pairwise interaction	a 'fighting' person with an opponent within 1.1–1.4 body heights, sustained ≥ 0.6 s
Fall	individual + temporal	activity becomes 'falling'
Person down	temporal context	still lying ≥ 6 s after a fall (no recovery)
Chasing	pairwise interaction	two fast movers, same heading, the follower moving towards the leader for ≥ 1.5 s
Group gathering	group interaction	≥ 4 stationary people linked within 1.2 body heights (single-linkage clustering)
Crowd gathering	group	≥ 8 people in view
Panic running	group	≥ 3 people running at the same time
Abandoned object	object-person relation	a bag whose owner (nearest person when it appeared) is > 2 body heights away for ≥ 15 s
After-hours presence	temporal context	anyone present during the configured quiet hours (22:00–06:00)

Table 3 – Suspicious and abnormal situations recognised by the scene-understanding module.

A scene summary is also written in natural language for the dashboard, for example: "3 people in view (2 male, 1 female, 0 undetermined) – 1 standing, 1 falling, 1 walking – observed: loitering, person down".

4.5 Real-time threat assessment and alert generation

Each SceneEvent gets a threat score between 0 and 100:

$$\text{score} = (\text{base}(\text{type}) + \text{duration bonus} + \text{people bonus} + \text{zone bonus} + \text{quiet-hours bonus} + \text{correlation bonus}) \times (0.7 + 0.3 \times \text{confidence})$$

Factor	Value
Base score	person down 85, altercation 80, fighting 75, panic running 70, chasing 65, abandoned object 65, restricted zone 60, fall 55, after-hours 40, loitering 35, group gathering 35, crowd 30, running 20
Duration bonus	type-specific rate (e.g. +1 per second of intrusion, +2 per second of fighting), capped at +15
People involved	+3 per person beyond two (group events), capped at +15
Zone sensitivity	+10 × zone sensitivity
Quiet hours	+15
Correlation	+20 fall after an altercation involving the same person; +10 zone entered while running; +10 chase linked to an altercation
Severity levels	LOW < 35 ≤ MEDIUM < 60 ≤ HIGH < 80 ≤ CRITICAL

Table 4 – Threat scoring model.

Alerts at MEDIUM level and above are raised (LOW events are shown only as overlays). To avoid flooding operators, the same situation — the same event type, people and zone — is not raised again within a 20-second cool-down while it continues. It is **escalated** (raised again, marked 'escalated') if its score rises by 10 points or it moves to a higher level; for example, an intrusion becomes CRITICAL after 10 seconds. Every alert keeps its list of reasons, so operators can see **why** it was raised.

For each alert the manager saves an annotated JPEG snapshot, writes the alert to the SQLite event history, pushes it to all open dashboards through Server-Sent Events (with a toast, a sound and a flashing banner for CRITICAL alerts), and forwards alerts at or above a configurable level by e-mail (SMTP with snapshot attached), a generic JSON webhook (Slack, Microsoft Teams or a SOC system) or Telegram. Notifications are sent from a background thread so they never slow down video processing.

4.6 Monitoring dashboard

The dashboard is a Flask web application with no external dependencies, so it works on an isolated security network. It has five pages:

- **Live monitor** – annotated MJPEG video (boxes coloured by activity or threat level, skeletons, zones, active threats), the scene description, KPI tiles (persons, male, female, undetermined, unique visitors, latency), an activity distribution, a per-person table (estimated gender with confidence, activity, dwell time, zone) and a live alert feed with acknowledgement.
- **Event history** – searchable and filterable table (severity, type, date range, text, unacknowledged only) with snapshot thumbnails, score explanations, pagination and CSV export.
- **Analytics** – alerts by severity, type and hour of day, plus occupancy and estimated gender over time (sampled every 5 s).
- **Performance metrics** – for every implemented algorithm: comparison tables with the best value highlighted, grouped bar charts of accuracy, precision, recall, F1-score and AP, per-class

and per-event-type F1, confusion matrices, speed charts and a live per-stage latency breakdown.

- **Sources & settings** – switch between the synthetic scene, webcams, video files and RTSP cameras, upload a video for analysis, and view the active configuration.

5. Implementation

5.1 Technology stack

Component	Technology
Language	Python 3.10+
Deep learning	PyTorch, Ultralytics YOLOv8 (pose + detection), torchvision (MobileNetV3)
Tracking	ByteTrack (Ultralytics), custom SORT-style IoU tracker (SciPy Hungarian)
Classical CV	OpenCV (video I/O, HOG+SVM, DNN module for Caffe face/gender models, drawing)
Web / real-time UI	Flask, MJPEG streaming, Server-Sent Events, vanilla JavaScript + SVG charts
Storage	SQLite (alerts, acknowledgements, occupancy statistics), JPEG snapshots
Configuration	YAML (zones, thresholds, scores, channels)
Testing	pytest unit and end-to-end tests

Table 5 – Technology stack.

5.2 Project structure

threatvision/	
├─ run.py	start pipeline + dashboard
├─ config.yaml	all parameters
├─ src/	
│ └─ detection/	YOLOv8-pose + ByteTrack, HOG baseline, simulated
detector	
├─ tracking/	IoU tracker, per-person TrackState
├─ gender/	face DNN, body CNN, hybrid
├─ activity/	features, rules, Bi-LSTM, TCN, dataset utilities
├─ scene/analyzer.py	interaction & context reasoning
├─ threat/	assessment (scoring) + alert manager (notifications)
├─ storage/db.py	SQLite event history
├─ pipeline/	processing thread, drawing, synthetic scene
└─ evaluation/metrics.py	accuracy, P/R/F1, confusion matrix, AP, event
matching	
├─ dashboard/	Flask app, templates, CSS, JS
└─ scripts/	download_models, extract_pose_dataset,
train_activity,	
└─ tests/	train_gender, prepare_pal00k, evaluate unit + end-to-end tests

5.3 Running the system

```

pip install -r requirements.txt
python scripts/download_models.py
python run.py --source simulated          # or: --source 0 | video.mp4 |
rtsp://...
# open http://localhost:5000

```

5.4 Synthetic self-test scene

For development and demonstration without a camera or trained models, a scripted 100-second synthetic scene renders stick figures and provides exact ground truth (boxes, keypoints, gender, activity). It includes walking and standing people, a loiterer, a two-person fight followed by a fall with no recovery, a restricted-zone intrusion, a chase, a bag left behind by its owner, and a group that gathers and then scatters in panic. The same scene drives the automated end-to-end test, which checks that every key threat type is raised.

6. Evaluation Methodology

6.1 Metrics

With TP, FP and FN the true positives, false positives and false negatives:

- **Precision** = $TP / (TP + FP)$: the share of detections or alerts that are correct (a measure of false alarms).
- **Recall** = $TP / (TP + FN)$: the share of real persons or events that are found (a measure of misses).
- **F1-score** = $2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$.
- **Accuracy** = correct predictions / all predictions (classification tasks).
- **AP@0.5** – area under the interpolated precision–recall curve for person detection at $\text{IoU} \geq 0.5$ (VOC all-point method).
- **Counting accuracy** – share of frames where the predicted person count equals the true count; **count MAE** – mean absolute counting error.
- **Processing speed** – frames per second, mean and 95th-percentile latency per frame, the real-time factor (processing FPS divided by source FPS), and latency per pipeline stage.

Multi-class results use **macro-averaging**, so rare but critical classes (falling, fighting) count as much as frequent ones. Confusion matrices are also reported.

6.2 Protocols and datasets

Module	Protocol	Recommended datasets
Person detection / counting	greedy $\text{IoU} \geq 0.5$ matching per image; P/R/F1 at the operating threshold; AP over all scores	COCO val2017 persons, CrowdHuman, COCO128 (included)
Gender	per-crop classification; coverage = share of crops where the model decides	PA-100K, PETA (body); Adience (face)
Activity	per-video test split; windows labelled by their last frame	Le2i / UR Fall, RWF-2000, Hockey Fight, KTH, own CCTV
Threat events	a prediction is TP if it falls inside an annotated interval of the same type (± 2 s); repeated alerts for an already-detected event are not counted as FP	annotated CCTV clips (JSON), UCF-Crime subsets, synthetic scene
Speed	300 frames after 10 warm-up frames, analytics only	any representative clip on the target hardware

Table 6 – Evaluation protocol. All metrics are computed by `scripts/evaluate.py` and shown on the dashboard.

7. Results and Discussion

Important: the results below come from two sources. Person detection was measured on **real images** (COCO128). Activity, threat-event and speed results were measured on the **synthetic self-test scene**; they show that the complete pipeline works, but they are **not** estimates of real-world accuracy. The rows marked 'pending' are filled automatically in results/metrics.json (and on the dashboard) when scripts/evaluate.py is run on the recommended real datasets with the deep models installed.

7.1 Person detection and counting (COCO128, 128 real images, 254 persons)

Algorithm	Precision	Recall	F1	AP@0.5	Count acc.	FPS (CPU)
HOG + linear SVM (baseline)	0.227	0.039	0.067	0.027	0.500	18.9
YOLOv8n-pose	pending					
YOLOv8s-pose	pending					

Table 7 – Person detection results (measured: HOG baseline; YOLOv8 rows: run scripts/evaluate.py detection).

The HOG baseline finds only 3.9 % of the people in COCO128 (10 of 254). COCO shows people sitting, partly occluded, at many scales and in crowds, which is far from the upright pedestrian template HOG was designed for. Its counting accuracy of 50 % comes mostly from images with no people in them. This supports the choice of a deep detector for the main pipeline. For reference, the publicly reported COCO pose mAP50 is about 0.80 for YOLOv8n-pose and 0.86 for YOLOv8s-pose [3]. The project's own measurement fills the table when the evaluation command is run.

7.2 Gender estimation

Algorithm	Accuracy	Precision	Recall	F1	Coverage	FPS
Face DNN (Levi-Hassner)	pending					
Body CNN (MobileNetV3)	pending					
Hybrid face → body	pending					

Table 8 – Gender estimation (run scripts/evaluate.py gender on the PA-100K test split).

Coverage matters as much as accuracy here. The face model is usually accurate when a face is found, but on CCTV it abstains for most people (low coverage). The body model covers everyone. Voting over time further improves the per-track result compared with the per-crop figures in the table.

7.3 Activity recognition

Algorithm	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	Windows / s
Rule-based (synthetic self-test)	0.888	0.929	0.837	0.867	49.5
Bi-LSTM + attention (real data)	pending				
TCN (real data)	pending				

Table 9 – Activity recognition. Rule-based result: 1467 windows from the synthetic scene.

Class	Precision	Recall	F1	Support
standing	0.875	0.962	0.916	624

Class	Precision	Recall	F1	Support
walking	0.875	0.855	0.865	558
running	1.000	0.516	0.681	93
fighting	0.925	0.896	0.910	96
falling	0.968	0.958	0.963	96

Table 10 – Per-class results of the rule-based classifier on the synthetic scene.

Falling (F1 0.96), standing (0.92) and fighting (0.91) are recognised reliably by the geometric rules. Running has perfect precision but only 0.52 recall: at the start of each running segment the average speed over 0.7 s is still below the running threshold, so those windows are labelled 'walking'. This transition lag is exactly what a learned sequence model is expected to reduce, because it can recognise the running gait from limb motion before the body reaches full speed. The LSTM and TCN are trained and compared with the same protocol on real footage (Section 4.3).

7.4 Suspicious-event and threat detection

Event type	Precision	Recall	F1	TP/FP/FN
abandoned object	1.00	1.00	1.00	2/0/0
chasing	1.00	1.00	1.00	2/0/0
fall	1.00	1.00	1.00	2/0/0
group gathering	1.00	1.00	1.00	2/0/0
loitering	1.00	1.00	1.00	2/0/0
panic running	1.00	1.00	1.00	2/0/0
person down	1.00	1.00	1.00	2/0/0
physical altercation	1.00	1.00	1.00	2/0/0
restricted zone intrusion	1.00	1.00	1.00	2/0/0

Table 11 – Event-level results on the synthetic scene (2 × 100 s). Overall precision 1.00, recall 1.00, F1 1.00.

All 18 scripted events (nine types, two cycles) were raised within the tolerance window, with no false alarms. Developing against this scene exposed several false-alarm patterns that were then fixed with contextual rules: people running side by side were first classified as fighting (fixed with the leg-motion cue), a passer-by near a bag reset the abandoned-object timer (fixed with owner tracking), and a bystander was paired with a fighter (fixed by requiring a passive partner to be very close). These cases show the value of explicit interaction reasoning. Because the scene was designed together with the rules, the perfect score should be read as a functional check, not as a measure of real-world performance.

7.5 Real-time processing speed

Configuration	FPS	Mean latency (ms)	P95 latency (ms)	× real-time
Simulated ground truth + Rule-based (pose geometry + motion)	315.9	3.2	5.0	15.79
HOG + SVM (baseline) + Rule-based (pose geometry + motion)	20.3	49.3	62.3	1.01

Table 12 – End-to-end analytics speed (x86_64, 300 frames, analytics only (no video encoding)).

Without the detector, the analytics layers (gender voting, activity recognition, scene reasoning and threat scoring) take about 3 ms per frame, so the system's speed is set almost entirely by the detector. With the HOG baseline on two CPU cores, the pipeline runs at about 20 fps, which is real time for a 20 fps stream. YOLOv8n-pose typically runs at 20–40 fps on a modern laptop CPU and at more than 100 fps on a mid-range NVIDIA GPU, which allows several cameras per GPU. The dashboard shows the live per-stage latency so bottlenecks can be spotted during deployment.

7.6 Dashboard

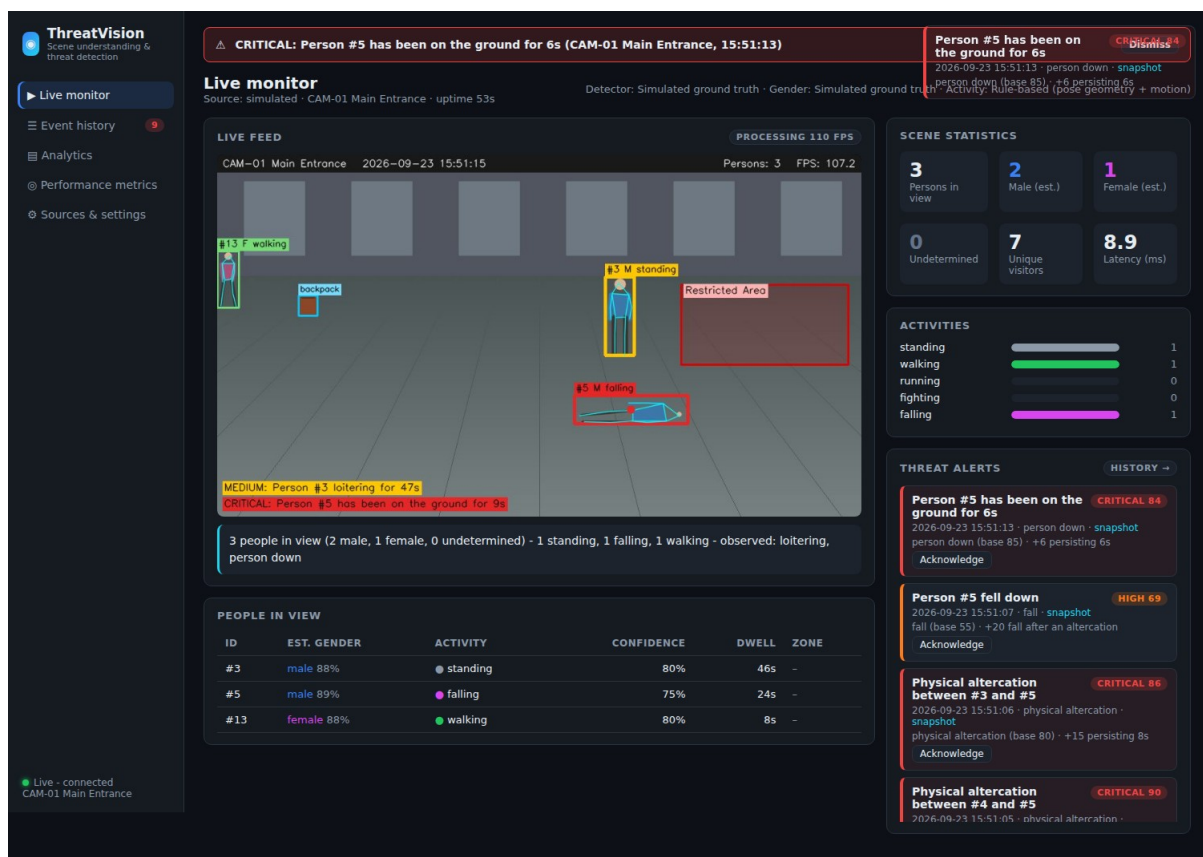


Figure 2 – Live monitor during the 'person down after an altercation' scenario: CRITICAL banner, annotated video, scene description, statistics and alert feed.

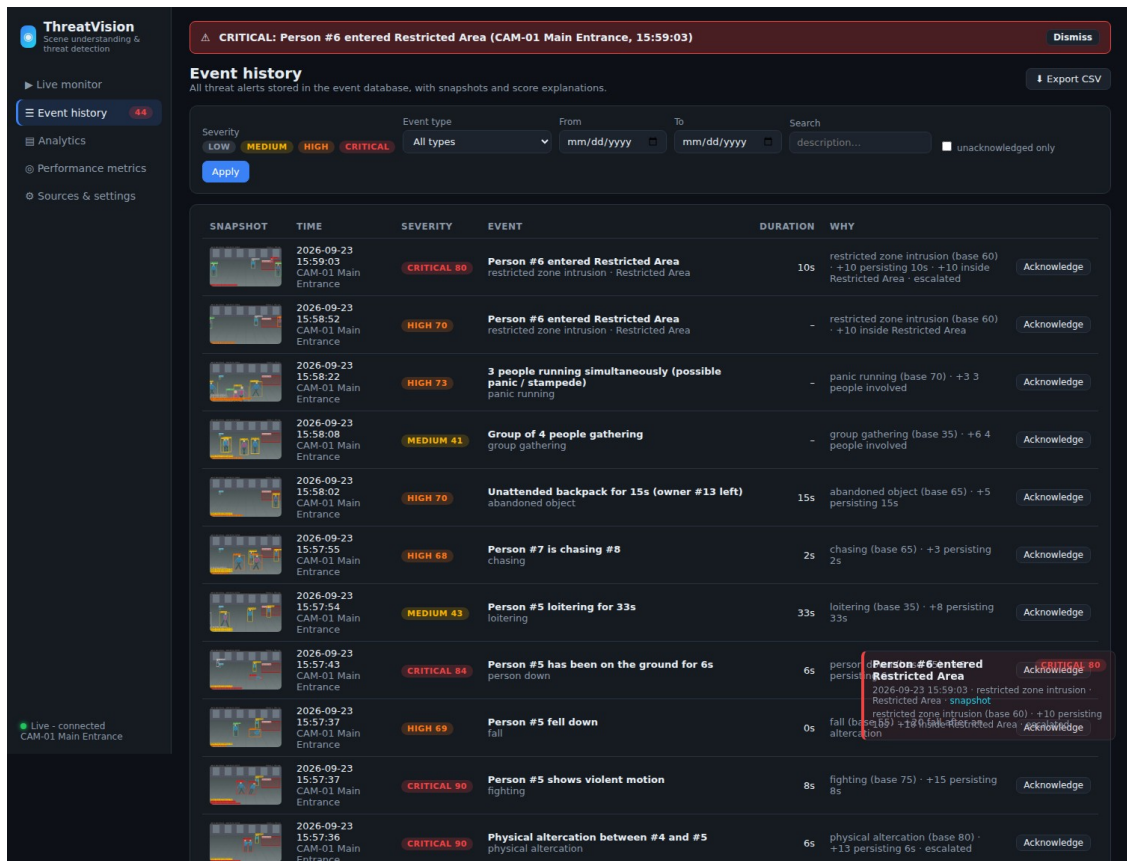


Figure 3 – Event history with severity filters, snapshots and score explanations.



Figure 4 - Performance-metrics page comparing the implemented algorithms (accuracy, precision, recall, F1, AP, speed, confusion matrices).



Figure 5 – Analytics: alerts by severity, type and hour, and occupancy and estimated gender over time.

8. Ethical, Legal and Privacy Considerations

- **Gender is an estimate.** It is inferred from appearance, is binary and probabilistic, and can be biased across demographic groups. It is shown only as an aggregate statistic with its confidence, reported as 'undetermined' when uncertain, and **never used in threat scoring**.
- **Human in the loop.** The system supports decisions; it does not make them. Every alert needs acknowledgement by an operator and explains its score.
- **Data protection.** Deployments must comply with applicable law (for example India's Digital Personal Data Protection Act 2023, or the GDPR). This means signage, purpose limitation, retention limits for snapshots and the event database, access control on the dashboard, and encrypted transport (HTTPS / VPN) for remote access.
- **Skeleton-based activity recognition** relies on body keypoints rather than identity, which lowers privacy risk and reduces dependence on clothing or skin tone.
- **Bias and robustness testing** on the target site's footage should be done before deployment, reporting metrics for each lighting condition and camera.

9. Limitations and Future Work

- Real-world accuracy of the learned activity models and the threat layer still has to be measured on annotated footage from the target site. The evaluation toolkit and data formats for this are provided.
- Single-camera reasoning: multi-camera re-identification would allow tracking across cameras and loitering detection over a whole site.
- Skeleton-based recognition cannot see objects held in the hand. Adding weapon detection (knife or gun detectors) and an ST-GCN [11] or video-transformer branch would extend coverage.

- Rule thresholds are set by hand. They could be tuned automatically on validation data, or replaced by a learned anomaly model (for example weakly supervised MIL [15]) with the rules kept as explanations.
- Edge deployment: export to ONNX or TensorRT for Jetson-class devices, and add a message bus (MQTT or Kafka) for multi-camera installations.

10. Conclusion

ThreatVision meets all six objectives. It counts people and estimates their gender with temporal voting. It recognises five human activities from pose sequences using three algorithms: interpretable rules, a Bi-LSTM with attention and a TCN. It reasons about interactions and context to identify eleven kinds of suspicious situation. It scores threats in real time with explainable, escalating severity levels and notifies personnel through several channels. It provides a complete monitoring dashboard, and it evaluates every algorithm with standard metrics shown on that dashboard. The modular design, configuration file and reproducible evaluation toolkit make it straightforward to adapt the system to a new site and to benchmark it on that site's own footage.

References

- [1] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," CVPR, 2005.
- [2] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, real-time object detection," CVPR, 2016.
- [3] G. Jocher, A. Chaurasia and J. Qiu, "Ultralytics YOLOv8," 2023. <https://github.com/ultralytics/ultralytics>
- [4] A. Bewley, Z. Ge, L. Ott, F. Ramos and B. Upcroft, "Simple online and realtime tracking," ICIP, 2016.
- [5] Y. Zhang et al., "ByteTrack: Multi-object tracking by associating every detection box," ECCV, 2022.
- [6] G. Levi and T. Hassner, "Age and gender classification using convolutional neural networks," CVPR Workshops, 2015.
- [7] X. Liu et al., "HydraPlus-Net: Attentive deep features for pedestrian analysis," ICCV, 2017 (PA-100K dataset).
- [8] A. Howard et al., "Searching for MobileNetV3," ICCV, 2019.
- [9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, 9(8), 1997.
- [10] S. Bai, J. Z. Kolter and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," arXiv:1803.01271, 2018.
- [11] S. Yan, Y. Xiong and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," AAAI, 2018.
- [12] I. Charfi, J. Miteran, J. Dubois, M. Atri and R. Tourki, "Optimized spatio-temporal descriptors for real-time fall detection," Journal of Electronic Imaging, 22(4), 2013 (Le2i dataset).
- [13] B. Kwolek and M. Kepski, "Human fall detection on embedded platform using depth maps and wireless accelerometer," Computer Methods and Programs in Biomedicine, 117(3), 2014 (UR Fall dataset).
- [14] M. Cheng, K. Cai and M. Li, "RWF-2000: An open large scale video database for violence detection," ICPR, 2021.
- [15] W. Sultani, C. Chen and M. Shah, "Real-world anomaly detection in surveillance videos," CVPR, 2018.
- [16] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," ECCV, 2014.

Appendix A – Evaluation commands

```
python scripts/evaluate.py detection --images data/eval/coco128 --algorithms
yolov8n-pose,yolov8s-pose,hog
```

```
python scripts/evaluate.py gender --images data/gender/test --algorithms  
face_dnn,body_cnn,hybrid  
python scripts/extract_pose_dataset.py --videos data/activity_videos --out  
data/activity/dataset.npz  
python scripts/train_activity.py --data data/activity/dataset.npz --model lstm  
python scripts/train_activity.py --data data/activity/dataset.npz --model tcn  
python scripts/evaluate.py activity --data data/activity/dataset.npz --algorithms  
rule_based,lstm,tcn  
python scripts/evaluate.py threat --videos data/eval/threat --annotations  
data/eval/threat/events.json  
python scripts/evaluate.py speed --video data/videos/clip.mp4 --configs  
yolo+rule_based,yolo+lstm,yolo+tcn,hog+rule_based
```